Density Estimation

Contents

5.1. Kernel Density Estimates 1
5.2 Bandwidth selection 7
5.3 Toy problem 11
5.4 Two-dimensional KDE 14
5.5 KDE for MCMC results 18
5.6 Homework #5 (10 points) 21

5.1. Kernel Density Estimates

This section follows the notes from Adrian Price-Wheelan

In probabilistic reasoning, it is imperative to have knowledge of the probability distributions governing the relevant random variables. Although a conceptual understanding of the distribution describing the problem at hand may be available, this information is often unknown. Hence, the central task in statistical inference is to estimate the underlying density distribution given a finite set of observations. These observations are inherently finite, prone to noise, and at times incompletely reported.

In a broader context, the crux of nearly all statistical inference problems involves the estimation of probability density functions for certain parameters or the computation of integrals over these probability densities. In situations where our models are parametric, algorithms like Markov Chain Monte Carlo (MCMC) are employed to produce samples from density distributions pertaining to the model parameters. However, there are instances when we either (1) lack a model, (2) possess a parametric model that is purely numerical, or (3) aim to promptly visualize density based on discrete samples. In such cases, non-parametric density estimation becomes a viable alternative.

The histogram stands out as a powerful tool for visualization due to its speed and widespread familiarity in interpretation. Notably, the insights derived from examining a histogram frequently hinge significantly on the selection of bins, encompassing both the placement and width of the bins.

As an illustration, consider the generation of histograms for 1D point data. In the forthcoming plots, all histograms are constructed using the same set of points denoted by the variable x. However, various selections for both bin widths and bin positions are employed in each case:

```
! pip install numpy matplotlib scikit-learn scipy astropy
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (1.23.5)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (3.7.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-packages (1.2.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages (1.11.3)
Requirement already satisfied: astropy in /usr/local/lib/python3.10/dist-packages (5.3.4)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (4.44.3)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (1.4.5)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (23.2)
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (9.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (3.1.1)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (2.8.2)
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.3.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (3.2.0
Requirement already satisfied: pyerfa>=2.0 in /usr/local/lib/python3.10/dist-packages (from astropy) (2.0.1.1)
Requirement already satisfied: PyYAML>=3.13 in /usr/local/lib/python3.10/dist-packages (from astropy) (6.0.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.7->matplotlib
```

```
import astropy.coordinates as coord
from astropy.table import Table
import astropy.units as u

import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
from scipy.stats import norm
from scipy.special import logsumexp
from scipy.optimize import minimize
from sklearn.neighbors import KernelDensity
from sklearn.model_selection import GridSearchCV
```

```
np.random.seed(42)
x1 = np.random.normal(0.25, 0.1, size=200)
x2 = np.random.normal(1., 0.5, size=1000)
x = np.concatenate((x1, x2))
np.random.shuffle(x) # we'll rely on this later!

def true_x_density(x):
    return ((1/6)*norm.pdf(x, loc=0.25, scale=0.1) +
            (5/6)*norm.pdf(x, loc=1., scale=0.5))
```

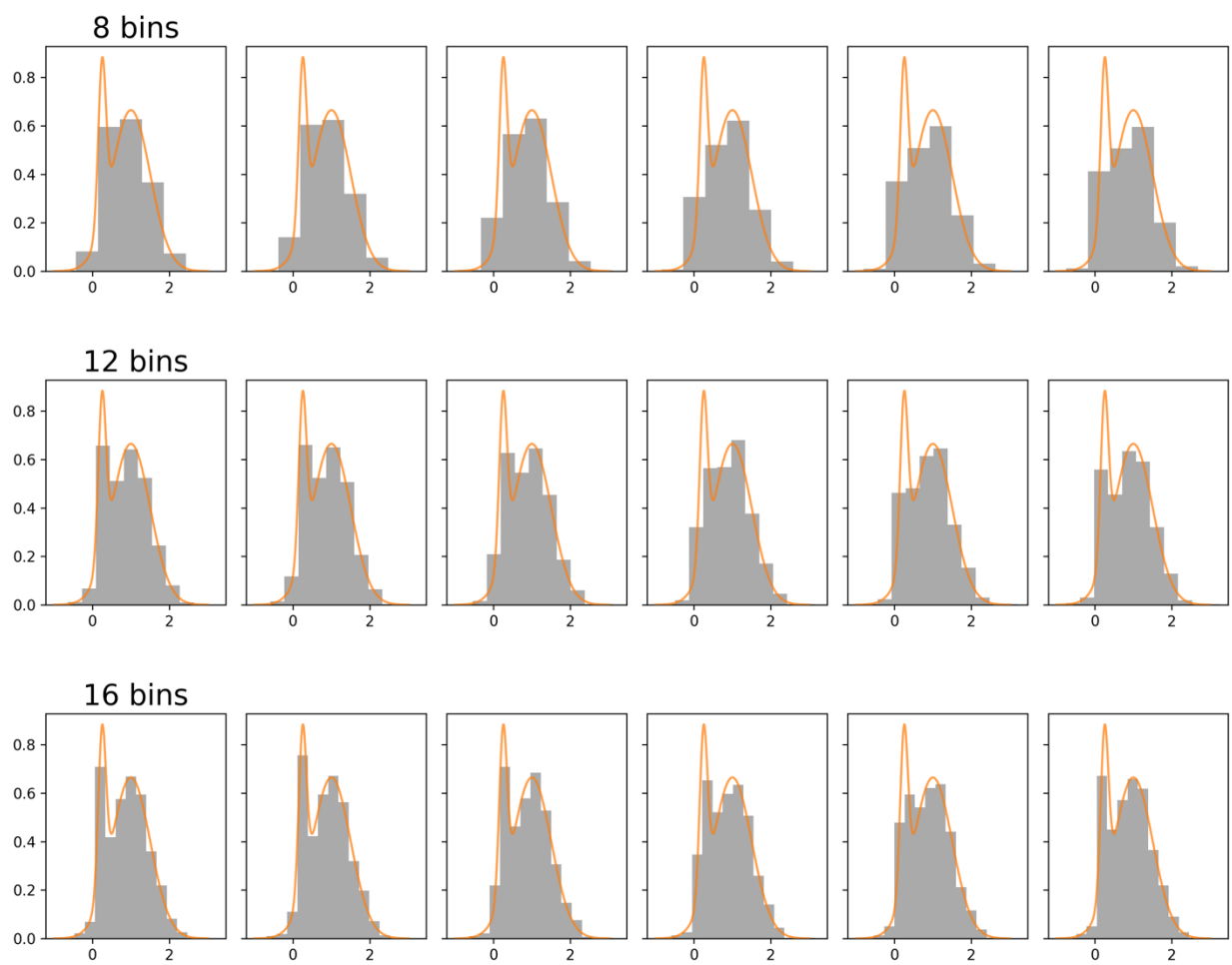
Left-to-right we shift all of the bins at a fixed number of bins, top-to-bottom we change the number of bins:

```
for n_bins in [8, 12, 16]:
    bins = np.linspace(-1, 3, n_bins)

    fig, axes = plt.subplots(1, 6, figsize=(12, 3),
                             sharex=True, sharey=True)

    x_grid = np.linspace(-1, 3, 1000)
    for i, ax in enumerate(axes.flat):
        ax.hist(x, bins=bins + i/20, density=True, color='#aaaaaa')
        ax.plot(x_grid, true_x_density(x_grid), marker='',
                alpha=0.75, zorder=10, color='tab:orange')

    axes[0].set_title('{0} bins'.format(n_bins), fontsize=20)
    fig.tight_layout()
```



Figures 5.1 – 5.3

Can we categorize the distribution above as unimodal or bimodal?

In a histogram, the bin positions and widths remain constant, and the density at each bin location is estimated by redistributing data points into respective bins and tallying counts. Alternatively, another approach to estimate density at a location involves maintaining fixed bin widths and data point locations, while adjusting the bin positions to overlap and align with each individual data point.

```
def point_centered_histogram(x, bin_width, data):
    data = np.array(data)
    x = np.array(x)
    y = (data[None] - x[:, None]) / (bin_width/2.)
    return np.sum(np.abs(y) < 1., axis=1) / (len(data) * bin_width)
```

Let's try this on some trivial data to start before applying it to the point data from above:

```
x_grid = np.linspace(0, 4, 512)
data = [1., 1.2, 3.]
bin_widths = [0.1, 0.3, 1.]

fig, axes = plt.subplots(1, 3, figsize=(9, 3), sharex=True, sharey=True)

for i, ax in enumerate(axes.flat):
    ax.plot(x_grid, point_centered_histogram(x_grid, bin_widths[i], data),
            marker='', drawstyle='steps-mid')

    ax.set_title('width={0:.1f}'.format(bin_widths[i]))
```

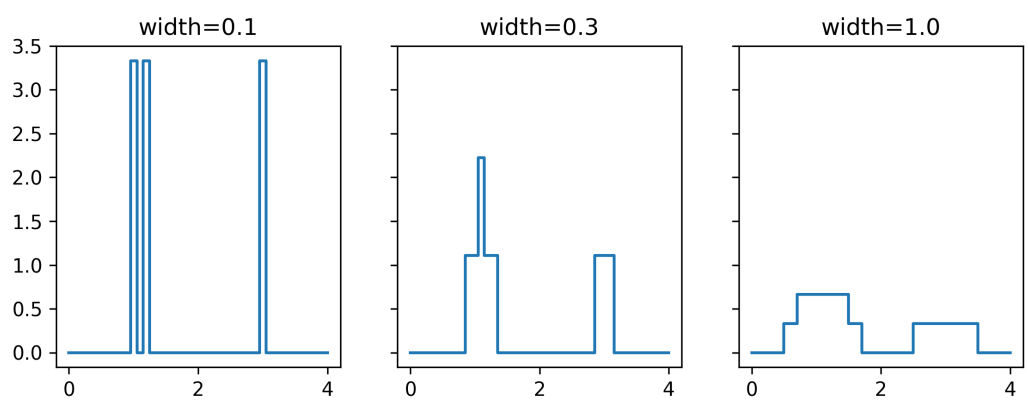


Figure 5.4

In the left panel, as the bin width diminishes, our "bins" are now centered directly on the data points. From left to right, with an increase in bin width, the bins are permitted to overlap. In this scenario, the necessity to explicitly define bin locations is eliminated, though specifying the bin

width remains essential. What has been implemented here is a kernel density estimation of the underlying density, utilizing a top-hat function as our kernel. Let's observe its performance with our earlier dataset:

```
fig, axes = plt.subplots(1, 6, figsize=(12, 3),
                         sharex=True, sharey=True)

bin_widths = np.logspace(-1, 1, len(axes))

x_grid = np.linspace(-1, 3, 1024)
for i, ax in enumerate(axes.flat):
    ax.plot(x_grid, point_centered_histogram(x_grid, bin_widths[i], x),
            marker='', drawstyle='steps-mid')

    ax.plot(x_grid, true_x_density(x_grid),
            zorder=-10, marker='')

    ax.set_title('width={0:.1f}'.format(bin_widths[i]), fontsize=20)
fig.tight_layout()

# Show the figure
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/PSU/ColabNotebooks/Example5_5.png',
dpi=300)
plt.show()
```

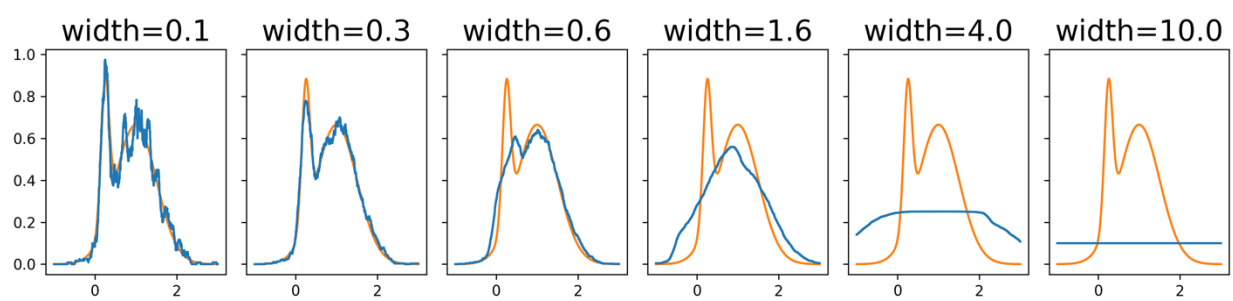


Figure 5.5

With a small bin size, the estimated density appears highly erratic. As the bin width, often referred to as the half-width or bandwidth, is increased, finer details in the data are progressively smoothed out, akin to the effect observed in a conventional histogram. Somewhere in the range of width=0.3–0.5, the density estimate exhibits a favorable resemblance to the true density (depicted in blue).

The top-hat serves as just one option among various available kernel functions. For the context of 1D data (expressions for higher-dimensional cases are available in other resources), the fundamental concept remains consistent: the density estimate at a given location, $\hat{f}(x)$, given a kernel function $K(z)$ and bandwidth b, is computed as follows:

$$\hat{f}(x) = \frac{1}{N}\frac{1}{b}\sum_{n}^{N} K\left(\frac{|x - x_n|}{b}\right)$$

where the N data points are indexed as x_n. Above, we used the top-hat kernel:

$$K(z) = 1 \; if \; z \leq 1 \; else \; 0$$

Other popular choices are the Gaussian kernel:

$$K(z) = \frac{1}{\sqrt{2\pi}} exp\left(-\frac{z^2}{2}\right)$$

Epanechnikov kernel:

$$K(z) = \frac{3}{4}(1 - z^2)$$

Scikit-learn provides implementations for all the aforementioned kernels. Let's leverage scikit-learn to construct Kernel Density Estimates (KDEs) for the aforementioned data, employing different kernels while keeping the bandwidth constant:

```
kde_tophat = KernelDensity(bandwidth=0.2, kernel='tophat')
kde_gauss = KernelDensity(bandwidth=0.2, kernel='gaussian')
kde_epan = KernelDensity(bandwidth=0.2, kernel='epanechnikov')
kdes = [kde_tophat, kde_gauss, kde_epan]

X = x.reshape(-1, 1)
for kde in kdes:
    kde.fit(X)

fig, axes = plt.subplots(1, 3, figsize=(9, 3),
                         sharex=True, sharey=True)

x_grid = np.linspace(-1, 3, 1024)
for ax, kde in zip(axes.flat, kdes):
    ax.plot(x_grid, np.exp(kde.score_samples(x_grid.reshape(-1, 1))),
            marker='', drawstyle='steps-mid')

    ax.plot(x_grid, true_x_density(x_grid),
            zorder=-10, marker='', color='tab:orange')
```

```
    ax.set_title(kde.kernel)

fig.tight_layout()

# Show the figure
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/PSU/ColabNotebooks/Example5_6.png',
dpi=300)
plt.show()
```

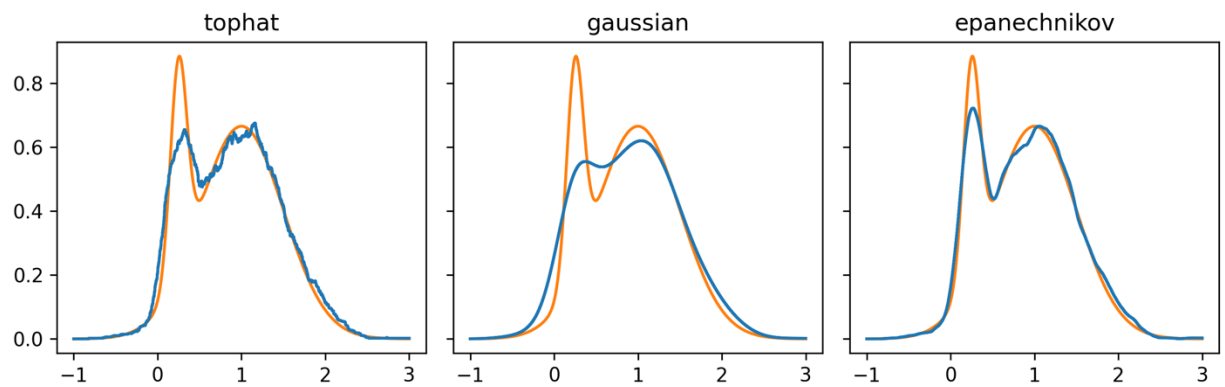


Figure 5.6

Surprisingly, when we use different methods to estimate density at the same level, the Gaussian approach doesn't work well in this case. Why? Because it has tails that go on forever. Although the Gaussian method can be good, choosing the right bandwidth is crucial for its success.

5.2 Bandwidth selection

With a histogram, we need to decide where the bins are and how wide they should be. Kernel density estimation is similar, but we also have to pick the bandwidth, which determines how smooth our density estimate turns out. This choice is super important because it affects our conclusions a lot. So, how do we choose the right bandwidth?

We use something called cross-validation (CV)!

In simple terms: we randomly set aside some data, create the density estimate, calculate how likely the set-aside data is under our estimate, and then adjust the bandwidth to maximize the overall likelihood.

To explain a bit more: cross-validation is a general way to check how well models work. It's commonly used in machine learning to fine-tune model settings. The basic idea is to split your data into two parts: a "training set" to build the model, and a "testing set" to see how well it predicts or represents new data. We repeat this process, adjusting our model each time, and then figure out the best overall setting. Usually, we just average the results from these adjustments.

But to do cross-validation, we need to decide:

1. How to split the data into training and testing?

2. How to score the models?

The answers depend on the specific problem. In our case, where we have individual data points and want to estimate the density using KDE, let's think about each question separately:

How can we divide our data into training and testing sets?

We have a few options:

1. Leave-one-out: Train the model on all data points except one, then test on the one left out. Repeat this for each data point.

2. Leave-K-out: Similar to leave-one-out, but now we leave out K data points and train on the rest. The testing set is the K points left out. This process is repeated until each point has been left out once.

3. K-fold: Divide the data into K equal-sized parts, train the model on $K-1$ of these parts, and test it on the remaining part. Repeat this process with different parts left out each time until all parts have been used for testing.

When we're estimating density, it's important to realize that as we take data points out of the training set (to put them in the test set), our density estimate becomes more erratic or noisy. A cautious approach in such cases is to use Leave-one-out cross-validation. In real-world scenarios, with ample data points, we might opt for Leave-K-out or K-fold cross-validation. However, if the dataset is small, we need to think carefully about these considerations!

How do we evaluate the models?

When it comes to cross-validation, another critical aspect is determining how to score a specific parameter choice or model. The scoring function is often referred to as a loss or cost function, as our goal is usually to minimize it. A common selection is the mean-squared error, which calculates the average squared difference between a test set value (what we observe) and the model's prediction for that value:

$$MSE = \frac{1}{N}\sum_{n}^{N}(\hat{y}_n - y_n)^2$$

For many problems, it's a good idea to test different cost functions. In our case, a straightforward choice is to use the density calculated by the Kernel Density Estimation (KDE) - essentially, the likelihood - as the cost function. However, when minimizing, it's crucial to remember to assess the negative likelihood.

Now, let's create a straightforward implementation of this cost function and apply leave-K-out cross-validation:

```
def cv_neg_ln_likelihood(p):
    ln_b = p[0]
    kde_gauss = KernelDensity(bandwidth=np.exp(ln_b), kernel='gaussian')
    kde_gauss.fit(train_x.reshape(-1, 1))
    return -np.sum(kde_gauss.score_samples(test_x.reshape(-1, 1)))
```

In our leave-K-out CV, we'll use K=32

```
bs = []

idx = np.arange(x.size)

# We'll only do 128 iterations, instead of running through the entire
dataset!
for i in range(128):
    rand_i = np.random.randint(0, x.size, size=32) # K = 32
    test_x = x[rand_i]
    train_x = x[np.logical_not(np.isin(idx, rand_i))]

    res = minimize(cv_neg_ln_likelihood, x0=-1)
    bs.append(np.exp(res.x[0]))

best_b = np.mean(bs)
best_b
```

```
0.09155566461581388
```

```
kde_gauss = KernelDensity(bandwidth=best_b, kernel='gaussian')
kde_gauss.fit(x.reshape(-1, 1))

plt.plot(x_grid, np.exp(kde_gauss.score_samples(x_grid.reshape(-1, 1))),
         marker='', drawstyle='steps-mid')
plt.plot(x_grid, true_x_density(x_grid),
         zorder=-10, marker='', color='tab:orange')
plt.title('Gaussian: b = {0:.2f}'.format(best_b))
```

```
# Show the figure
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/PSU/ColabNotebooks/Example5_7.png',
dpi=300)
plt.show()
```

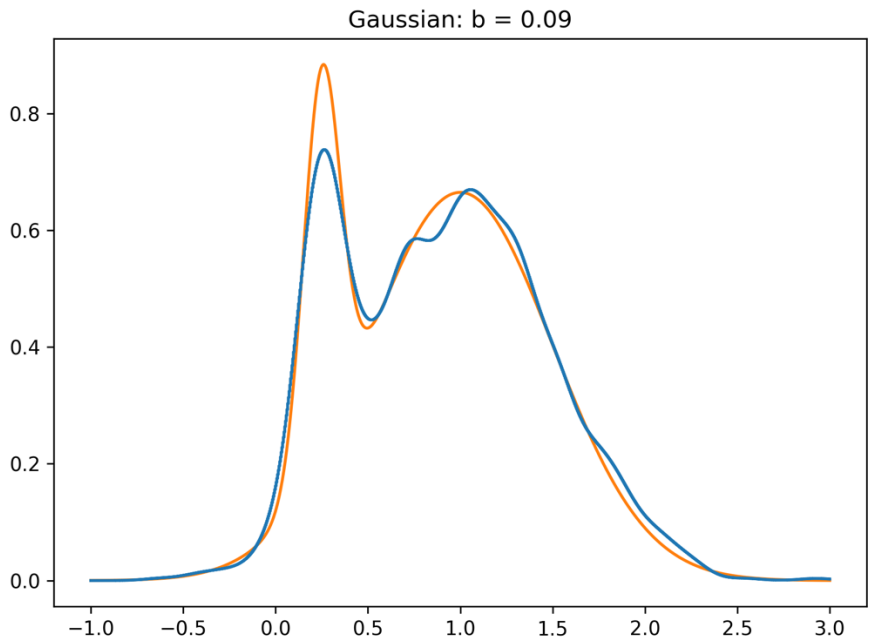


Figure 5.7

Scikit-learn also has functionality to do CV (via a grid search):

```
params = {'bandwidth': np.logspace(-3, 1, 128)}
grid = GridSearchCV(KernelDensity(), params)
grid.fit(x.reshape(-1, 1))

gridsearch_b = grid.best_params_['bandwidth']
kde_gauss = KernelDensity(bandwidth=gridsearch_b,
                          kernel='gaussian')
kde_gauss.fit(x.reshape(-1, 1))

plt.plot(x_grid, np.exp(kde_gauss.score_samples(x_grid.reshape(-1, 1))),
         marker='', drawstyle='steps-mid')
plt.plot(x_grid, true_x_density(x_grid),
         zorder=-10, marker='', color='tab:orange')

plt.title('Gaussian: b = {0:.2f}'.format(gridsearch_b))
```

```
# Show the figure
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/PSU/ColabNotebooks/Example5_8.png',
dpi=300)
plt.show()
```

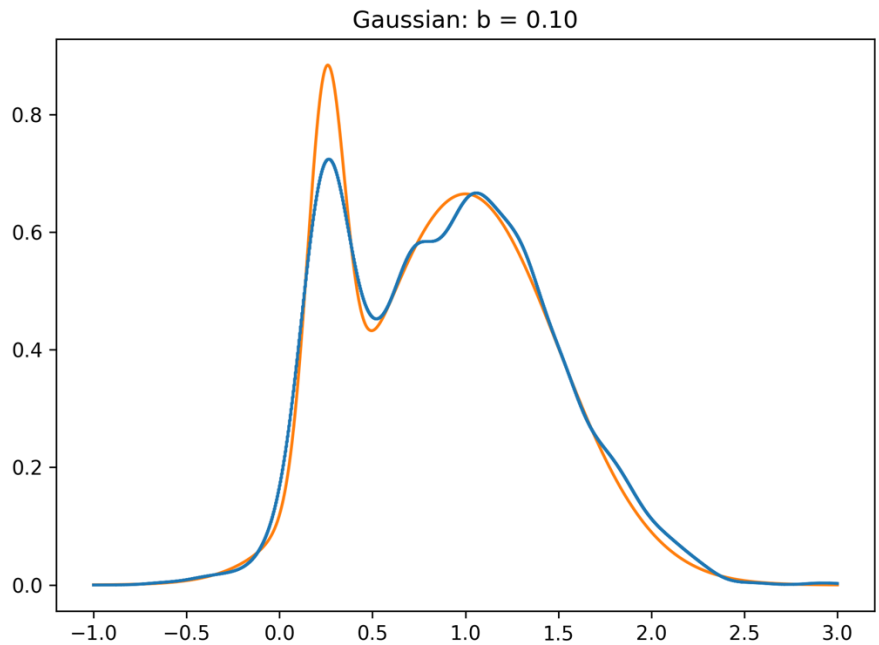


Figure 5.8

5.3 Toy problem

Now, let's explore using density estimation for classifying galaxies, combining KDE with Naive Bayes.

Imagine we've measured extremely accurate distances to a set of galaxies, neglecting uncertainties. Additionally, our selection process is uniform across the entire volume. Among these galaxies, some are part of a galaxy cluster, while others belong to the broader field population. We have additional data that provides labels for these two classes, meaning we know which galaxies are in the cluster and which ones are in the field.

```
tbl = Table.read('/content/drive/MyDrive/PSU/ColabNotebooks/galaxies.csv')

dist_bins = np.linspace(450, 550, 31)

for cls in np.unique(tbl['label']):
```

```
    plt.hist(tbl['dist'][tbl['label'] == cls],
             bins=dist_bins, alpha=0.4, label=cls)

plt.legend(fontsize=16)

# Show the figure
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/PSU/ColabNotebooks/Example5_9.png',
dpi=300)
plt.show()
```

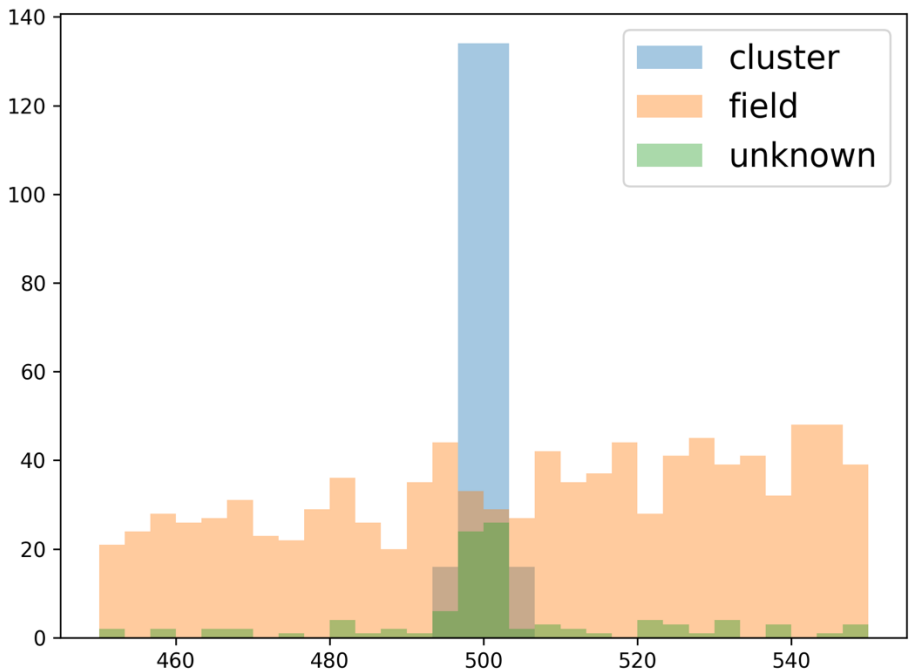


Figure 5.9

Let's build density estimates of the "cluster" and "field" populations:

```
params = {'bandwidth': np.logspace(-3, 1, 128)}

kdes = dict()
for label in ['cluster', 'field']:
    cv = GridSearchCV(KernelDensity(), params)
    cv.fit(tbl['dist'][tbl['label'] == label].reshape(-1, 1))

    kdes[label] = KernelDensity(bandwidth=cv.best_params_['bandwidth'])
    kdes[label].fit(tbl['dist'][tbl['label'] == label].reshape(-1, 1))

n_cluster = (tbl['label'] == 'cluster').sum()
```

```
n_field = (tbl['label'] == 'field').sum()
print('{0} in cluster, {1} in field'.format(n_cluster, n_field))

weights = dict()
weights['cluster'] = n_cluster / (n_cluster + n_field)
weights['field'] = n_field / (n_cluster + n_field)
weights
```

300 in cluster, 1000 in field

{'cluster': 0.23076923076923078, 'field': 0.7692307692307693}

```
unknown = tbl[tbl['label'] == 'unknown']

norm = []
for label in weights.keys():
    ln_likelihoods = kdes[label].score_samples(unknown['dist'].reshape(-1,
1))
    norm = np.sum(weights[label] * np.exp(ln_likelihoods))
norm = np.sum(norm)

prob_field = weights['field'] *
np.exp(kdes['field'].score_samples(unknown['dist'].reshape(-1, 1))) / norm
prob_cluster = weights['cluster'] *
np.exp(kdes['cluster'].score_samples(unknown['dist'].reshape(-1, 1))) /
norm
ratio = prob_cluster / prob_field

unknown_labels = np.full(len(unknown), 'kasdfkjfsaj')
unknown_labels[ratio > 1] = 'cluster'
unknown_labels[ratio <= 1] = 'field'

dist_bins = np.linspace(450, 550, 31)

for cls in np.unique(tbl['label']):
    plt.hist(unknown['dist'][unknown_labels == cls],
             bins=dist_bins, alpha=0.4, label=cls)

# Show the figure
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/PSU/ColabNotebooks/Example5_10.png',
dpi=300)
plt.show()
```

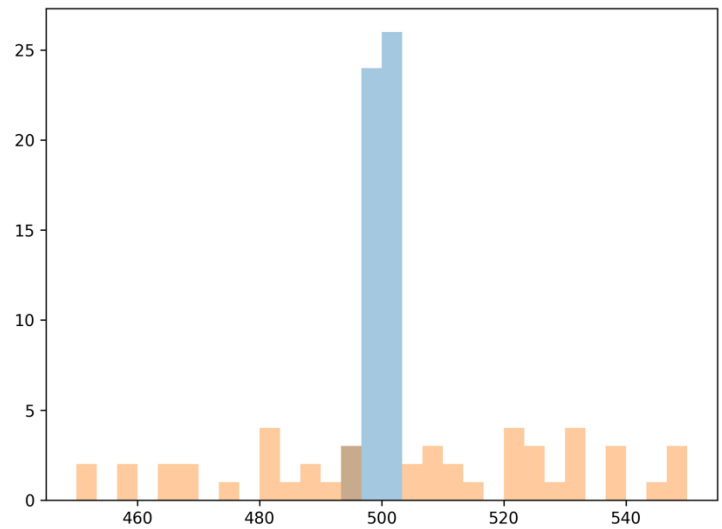


Figure 5.10

5.4 Two-dimensional KDE

So far, we have only done KDE on one-dimensional data, but the algorithm generalizes easily to higher dimensions. Let's see how the 2D version works:

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as st

# Generating synthetic hydrological data
np.random.seed(42)
discharge = np.random.normal(loc=50, scale=10, size=100)  # Example
discharge data (cubic meters per second)
precipitation = np.random.normal(loc=20, scale=5, size=100)  # Example
precipitation data (millimeters)

# Combine discharge and precipitation into a 2D array
hydro_data = np.column_stack((discharge, precipitation))

# Extract the two hydrological variables
x = hydro_data[:, 0]
y = hydro_data[:, 1]

# Set the plot limits
xmin, xmax = min(x), max(x)
ymin, ymax = min(y), max(y)
```

```
# Perform the kernel density estimate
xx, yy = np.mgrid[xmin:xmax:100j, ymin:ymax:100j]
positions = np.vstack([xx.ravel(), yy.ravel()])
values = np.vstack([x, y])
kernel = st.gaussian_kde(values)
f = np.reshape(kernel(positions).T, xx.shape)

# Plotting
fig, ax = plt.subplots()
ax.set_xlim(xmin, xmax)
ax.set_ylim(ymin, ymax)

# Contourf plot
cfset = ax.contourf(xx, yy, f, cmap='Blues')
## Or kernel density estimate plot instead of the contourf plot
ax.imshow(np.rot90(f), cmap='Blues', extent=[xmin, xmax, ymin, ymax])

# Contour plot
cset = ax.contour(xx, yy, f, colors='k')

# Label plot
ax.clabel(cset, inline=1, fontsize=10)
ax.set_xlabel('River Discharge (m³/s)')
ax.set_ylabel('Precipitation (mm)')

# Show the figure
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/PSU/ColabNotebooks/Example5_11.png',
dpi=300)
plt.show()
```

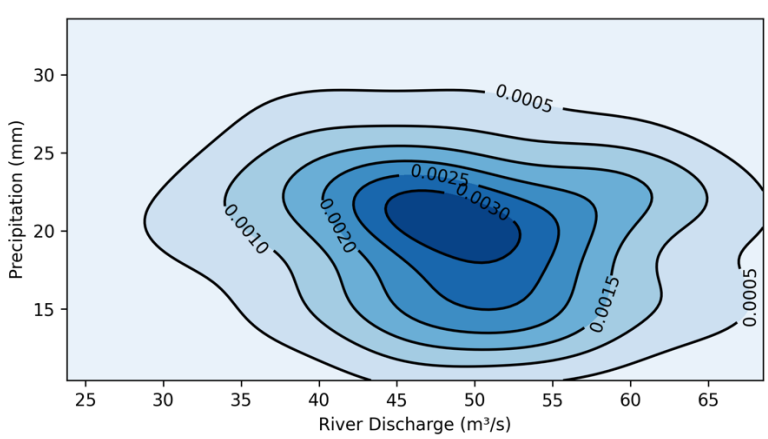


Figure 5.11

```
df_finch = pd.read_csv('/content/drive/MyDrive/PSU/ColabNotebooks/darwin
finch data.csv')
print(df_finch.head())

# parent_beak_depth_mm
mean_parent_BD = df_finch['parent_beak_depth_mm']

# set up grids. Borrowed from scipy.stats.gaussian_kde documentation.
X, Y = np.mgrid[6:12:100j, 6:12:100j]
positions = np.vstack([X.ravel(), Y.ravel()])
values = np.vstack([df_finch['offspring_beak_depth_mm'], mean_parent_BD])
kernel = st.gaussian_kde(values)
Z = np.reshape(kernel(positions).T, X.shape)

fig, ax = plt.subplots()
_ = ax.plot(df_finch['offspring_beak_depth_mm'], mean_parent_BD,'k.',
alpha=0.6)
_ = ax.imshow(np.rot90(Z), extent=[6, 12, 6, 12], cmap='Greens')
_ = plt.xlabel('Offspring beak depth (mm)')
_ = plt.ylabel('Mean parental beak depth (mm)')

# Show the figure
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/PSU/ColabNotebooks/Example5_12.png',
dpi=300)
plt.show()
```

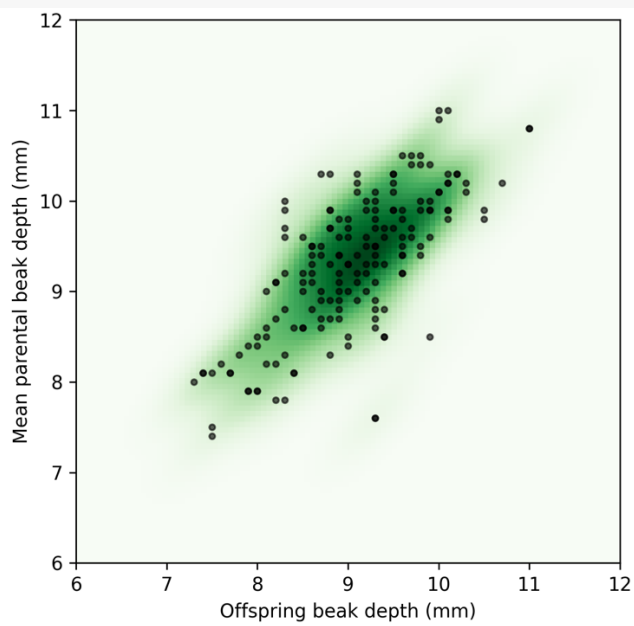


Figure 5.12

```
import seaborn as sns
import matplotlib.pyplot as plt

# Assuming df_finch is your DataFrame

# Set the figure size
fig, ax = plt.subplots()

# Create a Seaborn KDE plot with contour lines (no fill)
sns.kdeplot(x=df_finch['parent_beak_depth_mm'],
y=df_finch['offspring_beak_depth_mm'], ax=ax, cmap="viridis")

# Add labels
ax.set_xlabel('Parent Beak Depth (mm)')
ax.set_ylabel('Offspring Beak Depth (mm)')

# Show the figure
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/PSU/ColabNotebooks/Example5_13.png',
dpi=300)
plt.show()
```

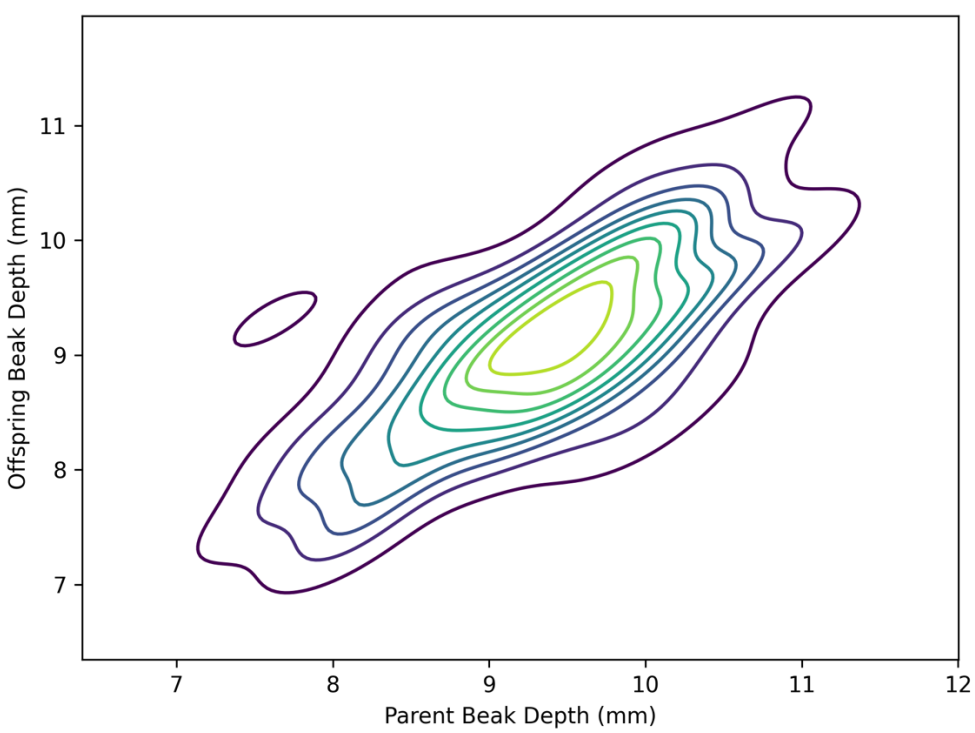


Figure 5.13

In theory, Kernel Density Estimation (KDE) can be applied to higher-dimensional data; however, in practice, this is not a common practice. This is partially due to the inherent difficulty in presenting and visualizing the results, but it is also attributed to the introduction of new complexities in higher dimensions. Unlike in lower dimensions, the bandwidth is no longer a singular parameter; instead, the kernel is characterized by a bandwidth matrix (consider an asymmetric kernel for the aforementioned finch data, for instance). Naturally, selecting the elements of this matrix can swiftly become a challenging problem as dimensionality increases. While algorithms sharing a conceptual resemblance to KDE are occasionally employed in high-dimensional machine learning scenarios, they typically operate under different names. Once again, it is important to note that delving into these advanced concepts is well beyond the scope of this course.

5.5 KDE for MCMC results

The following code demonstrates the use of Kernel Density Estimation (KDE) in the context of a Markov Chain Monte Carlo (MCMC) simulation. It generates synthetic data from a normal distribution, and then employs the `emcee` library for MCMC sampling to infer the parameters of the distribution (mean and standard deviation). The MCMC samples are collected, and a 2D KDE plot is created using the `seaborn` library to visualize the joint probability distribution of the inferred parameters. The resulting plot provides insights into the uncertainty and correlation between the estimated mean and standard deviation, showcasing the power of KDE in summarizing complex MCMC output. Adjustments to the data generation process and likelihood function can be made to suit specific modeling scenarios.

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import emcee  # You may need to install this package if not already
installed

# Generate synthetic data for demonstration purposes
np.random.seed(42)
data = np.random.normal(loc=3, scale=2, size=1000)

# Define log likelihood function (assuming a normal distribution)
def log_likelihood(theta, data):
    mu, sigma = theta
    model = np.random.normal(loc=mu, scale=sigma, size=len(data))
    return -0.5 * np.sum((model - data)**2)

# Set up the MCMC sampler
```

```
ndim = 2  # Number of parameters (mean and standard deviation)
nwalkers = 100
nsteps = 1000
initial_guess = [2, 1]  # Initial guess for mean and standard deviation

sampler = emcee.EnsembleSampler(nwalkers, ndim, log_likelihood,
args=[data])

# Run the MCMC sampler
sampler.run_mcmc(initial_guess + 1e-4 * np.random.randn(nwalkers, ndim),
nsteps)

# Extract the samples from the MCMC chain
samples = sampler.chain[:, 500:, :].reshape((-1, ndim))

# Plot the MCMC samples using KDE
sns.set(style="white", palette="muted")
sns.jointplot(x=samples[:, 0], y=samples[:, 1], kind="kde", space=0,
color="b")

plt.xlabel("Mean")
plt.ylabel("Standard Deviation")

# Show the figure
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/PSU/ColabNotebooks/Example5_14.png',
dpi=300)
plt.show()
```

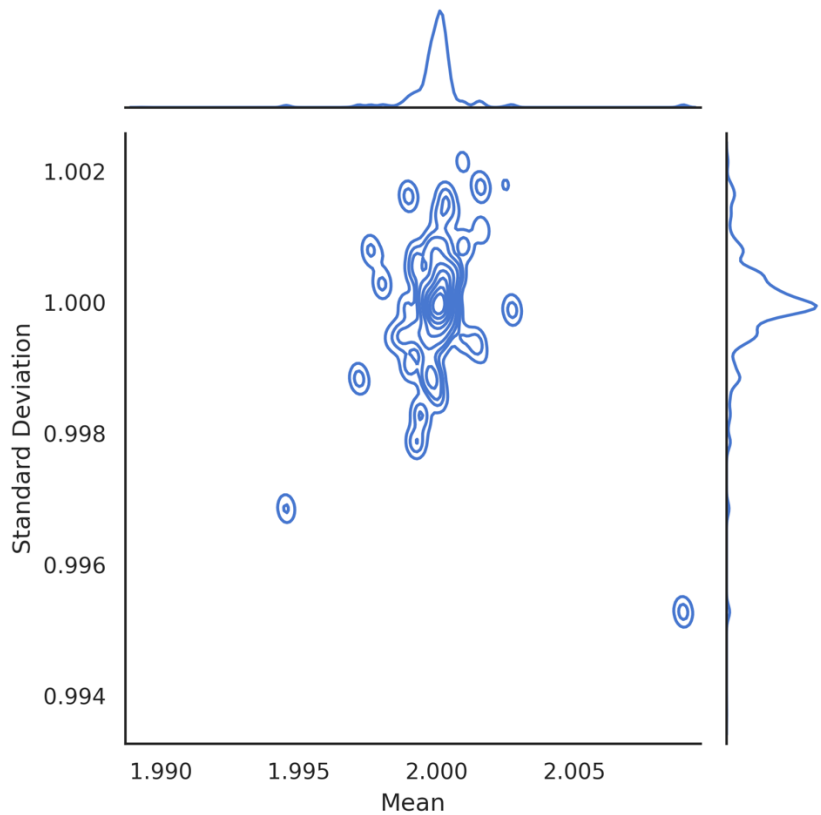


Figure 5.14

KDE offers several advantages beyond its application in Markov Chain Monte Carlo (MCMC) simulations:

1. Non-Parametric Estimation: KDE provides a non-parametric way to estimate the probability density function of a random variable without assuming a specific parametric form for the underlying distribution. This flexibility makes it applicable to a wide range of data types.
2. Smooth Representation: KDE produces a smooth estimate of the underlying distribution, helping to reduce the impact of noise in the data. This can be particularly useful for visualizing the overall shape of the distribution.
3. Visualization: KDE is a powerful tool for visualizing the distribution of data. It allows for a more informative representation compared to histograms, especially when dealing with continuous data.
4. Bandwidth Selection: The bandwidth parameter in KDE controls the smoothness of the estimate. Adaptive bandwidth selection methods can be employed to automatically adjust the bandwidth based on the characteristics of the data, ensuring optimal representation.
5. Handling Multi-Modal Distributions: KDE can effectively capture and visualize multi-modal distributions, which may be challenging with other methods like histograms.

6. Data Imputation: KDE can be used for imputing missing values in a dataset by estimating the conditional distribution of a variable given observed data.
7. Statistical Inference: KDE is used in statistical inference tasks, such as estimating the cumulative distribution function, quantiles, and conducting hypothesis tests about the underlying distribution.
8. Machine Learning: KDE-inspired techniques are employed in machine learning algorithms, including density estimation in clustering and anomaly detection tasks.
9. Cross-Validation: KDE allows for the use of cross-validation techniques to optimize bandwidth selection, helping to prevent overfitting or underfitting of the data.
10. Mixture Model Initialization: In unsupervised machine learning, KDE can be used for initializing parameters in mixture models, enhancing the efficiency of algorithms like the Expectation-Maximization (EM) algorithm.
These advantages highlight the versatility and utility of KDE in various statistical and data analysis applications.

5.6 Homework #5 (10 points)

Imagine you are a climate scientist studying heatwaves, and you are interested in understanding the factors influencing the duration of heatwaves. You decide to conduct a simulation to estimate parameters for a model that predicts heatwave durations based on average daily temperature and humidity. The model is assumed to follow a simple linear relationship.

Data Generation: Generate synthetic data for 1000 days, including average daily temperature (in degrees Celsius) and humidity (in percentage). For "average daily temperature", use uniform distribution to sample data, where the low and high values are, 25 and 40, respectively. For "humidity", sample data from a normal distribution with mean and standard deviation of 50, and 10, respectively. Make sure to use *np.random.seed(XX).* XX is the last two digits of your PSU ID.

Use the following to generate noisy heatwave durations:

```
# True parameters for generating synthetic data
true_intercept = 5
true_slope = 0.2
true_sigma = 2

# Simulate noisy heatwave durations
noise = np.random.normal(loc=0, scale=true_sigma, size=1000)
heatwave_durations = true_intercept + true_slope * avg_daily_temperature +
noise
```

1. MCMC Simulation: Use the `*emcee*` library to set up an MCMC simulation. Assume a linear regression model for simplicity and set up the log likelihood function. The goal is to estimate the parameters (`*intercept*`, `*slope*`, and `*sigma*`) that best describe the relationship between average daily temperature and heatwave durations.

2. KDE Visualization: After running the MCMC simulation, extract the samples from the chain and use Kernel Density Estimation (KDE) to visualize the joint distribution of the intercept and slope parameters. Plot a 2D KDE plot to gain insights into the relationships between temperature, humidity, and heatwave durations.

3. Answer the following questions based on the KDE plot:
- What does the joint distribution reveal about the impact of average daily temperature on heatwave durations?
- How does humidity factor into the relationship, as indicated by the KDE plot?
- Can you identify any patterns or clusters in the joint distribution, and what insights do they provide about the model parameters?

Provide your Python script for data generation, MCMC simulation, and KDE visualization.